NOCKONOMICON

N. E. Davis

Contents

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

> Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.
>
> (Keiji Nishitani, *Religion and Nothingness*, 1982, pp. 129–130)

There are many worthy raconteurs of the history of computing, such as **Hofstadter1980** (**Hofstadter1980**) and **Penrose2004** (**Penrose2004**). What distinguishes my approach to the history of computing from other tellings? Computing has deep connexions to alchemy and true language, which I hope to demonstrate without assuming any Hermetic background on the reader's part. Much like technology, computing is not merely a tool but a stance towards the world. However, it does not entail technologism's supercession; computing is always both a lens and a kaleidoscope, through which many layers of reality can be seen simultaneously. It is a way of seeing the world as a system of pure statements and acting to make those statements true. It is both ontology and epistemology.

First, therefore, I must establish that computation is the apex of a long human search for a language of truth. The origin of symbol itself in the σύμβολον, a

N. E. Davis ~lagrev-nocfep. "The Roots of Computing in the Ancient World". *The Nock Book* (2025): 1–x.

token broken in half to infallibly mark identity, is one font. Later "symbol" was the profession of faith, both an attempt to formulate truth and a sign of shared claims. The primary source, however, is language itself, Hofstadter's "strange loop" of self-reference. The first possible Nock formula is a crash, the void. The second is self-reference, awakening. Peirce would be proud of the progression. We have tricked ourselves into thinking that computing is a late development, a fruitful elaboration of the end of mathematics. In fact, it is the philosopher's stone, and it underlies every process in the world. It has turned lead into gold.

The historical account which follows aims throughout at the goal of understanding what Nock is and why it matters as a protocol. The reader will understand what mankind has been looking for, when we have caught various pieces of the puzzle, how these things came together in the mid-20th century, and how they have been refined into a pure machine.

Outline: - Cuneiform, tabulation, calculation, and algorithm - Solid-state indices: abaci, tokens, etc. Mechanism as embodied equation. - Pure language and true statements (Llull, Leibniz, Wilkins) - Generalization (Jacquard, Babbage) - The universal machine (Turing, Gödel, Zuse, von Neumann) - Pure statements as Pages from the Book (Curry, Church)

1.1 Tabulation

1.2 Solid-state indices

The problem is that any calculation lives in someone's head or someone's claim. This is a summary, but not the proof or solution.

Chapter 2

Combinators for Fun and Profit

Abstract

The first section of this chapter will introduce the basic SKI and BCKW combinator calculi and examine how they can represent each other. The second section will introduce Nock's opcodes as combinators, and show how they can be used to express various combinator idioms. The third section will show how to build practical affordances for common programming tasks, such as conditionals, recursion, subprocedure invocation, and data structures. The final section will discuss the relationship between combinators and virtualization, and how Nock can be used to build a virtual machine even in a crash-only context.

2.1 Combinators in Computing

The pioneers of computing elaborated three complementary visions:

1. The Turing machine, a simple abstract machine that can compute anything computable.
2. The lambda calculus, a pure functional language based on substitution and abstraction.
3. Combinatory logic, a variable-free functional language based on combinators.

Logicians early demonstrated the equivalence of these approaches, but particular formulations are still preferred for particular tasks. The Turing machine is the most intuitive model of computation, and is often used to prove computability results. The lambda calculus is the basis of many functional programming

languages, such as Lisp, Scheme, and Haskell. Combinatory logic is less widely used in practice, but has a long history in mathematical logic and type theory. It is also the basis of Nock.

Before we get too far out over our skis, let's talk about what a combinator is. At its simplest, a combinator is a function that takes one or more arguments and returns a result, without referring to any variables outside its own arguments. In practice, we'll use different combinators to build up more complex functions, and we'll see informally how they can be combined to express any computable function. We can even build combinators out of other combinators. The objective of combinatory logic is to express all computation in terms of combinators alone, without the need for variables or named functions.

This first section will introduce some elemental and compound combinators and make the case that they circumscribe "computing" as a concept.

2.1.1 The SKI calculus

Computable functions can be expressed using minimal sets of combinators. Many logicians today prefer to work with Moses Schönfinkel and Haskell Curry's SKI combinator calculus, which has three combinators:

The identity combinator I is conventionally written `Ix = x`. This simply means that it returns its single argument unchanged. There is little to say of this combinator at this point, but logicians have found that it frequently simplifies expressions.

```
I x == x
```

The constant combinator K is conventionally written `Kxy = x`. This means that it takes two arguments and returns the first, ignoring the second. It is useful for building functions that always return a fixed value.

Questions

1. What is the result of `I x`?
2. What is the result of `I I x`?

```
K x y == x
```

The substitution combinator S is conventionally written `Sxyz = xz(yz)`. This means that it takes three arguments: a function `x`, and two values `y` and `z`. It applies the function `x` to the value `z`, and also applies the function `y` to the value `z`, then applies the result of `x z` to the result of `y z`. This combinator is more

complex than the others, but it is very powerful, as it allows us to express function application and composition.

```
S x y z == x z (y z)
```

The SKI system is Turing complete (= μ-recursive), meaning that any computable function can be expressed using only these three combinators. It is also equivalent to the lambda calculus, meaning that any lambda expression can be translated into an equivalent SKI expression and vice versa (Kleene (1952), Theorem XXX).[1]

Combinators can be combined to form more complex expressions. For example, the combinator `S K K` is equivalent to the identity combinator `I`, which can be proven thus:

```
S K K z ==       ; x -> K, y -> K, z -> z
K z (K z) ==     ; K z -> z
z
```

In such cases, combinators are applied from left to right by the first possible reduction. This means that they will bottom out in a particular expression, which may be the argument (as with `I x == x`).

2.1.2 The BCKW calculus

The BCKW combinator calculus is an alternative to SKI, with a different set of primitive combinators. It has four combinators:

The composition combinator `B` is conventionally written `Bxyz = x(yz)`. This means that it takes three arguments: a function `x`, and two values `y` and `z`. It applies the function `y` to the value `z`, then applies the function `x` to the result.

```
B x y z == x (y z)
```

The flip combinator `C` is conventionally written `Cxyz = xzy`. This means that it takes three arguments: a function `x`, and two values `y` and `z`. It applies the

[1] Four approaches to general-purpose computing were devised around the same time: Moses Schönfinkel and Haskell Curry's combinator SKI calculus (1920s); Alonzo Church's lambda calculus λC (1936); Alan Turing's tape machine (1936); and Stephen Kleene's μ-recursive functions (1936) building on Kurt Gödel's primitive recursive (1931) and general recursive functions (1934). These have been demonstrated to be equivalent to each other by means of the Church–Turing thesis: "Every effectively calculable function is a computable function." Various auxiliary proofs establish the isomorphic nature of these systematic approaches; however, a footnote much larger than the length of this book is warranted to address the relationships amongst these four formulations. We simply take for granted their equivalence and move freely between representations as necessary. Furthermore, other formulations, such as the structured program theorem for control flow graphs (Böhm and Jacopini, 1966), exist and merit future exploration.

function x to the values z and y, effectively reversing the order of the last two arguments.

```
C x y z == x z y
```

The join combinator 𝕎 is conventionally written 𝕎xy = xyy. This means that it takes two arguments: a function x and a value y. It applies the function x to the value y twice.

```
𝕎 x y == x y y
```

The SKI and BCKW systems are equivalent, meaning that any expression in one system can be translated into an equivalent expression in the other system. For example, the identity combinator I can be expressed in BCKW as:

```
I = 𝕎 K
```

```
    U
    Y
```

```
Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))
```

```
    𝕄ABT or whatever it is
```

Chris Barker The iota combinator ɩ can produce all the others, but in convoluted ways.

Combinator calculi are generally considered too minimalist for practical programming; they shine, however, in mathematical proofs. This means that we can use them to define a standard of behavior for an algorithm, but implement the logic in a compliant form using whatever computational tools we like. Thus it is better to think of Nock not as a bare programming language, but as a standard of computation which can under many circumstances be used as the computation itself.[2]

2.2 Nock Opcodes as Combinator

At this point, we must engage Nock as a specification for computing. Nock defines a number of opcodes ("operation codes") which span a range of common functional behaviors, including those we have seen in the basic combinators. Nock represents all data as a noun, which is either an atom (a natural number, which may be thought of a Gödel number or raw data as bytes) or a cell (an ordered pair of nouns, with a head and a tail). Nock opcodes operate with at least two arguments: a "subject" which defines the operating environment, more or less,

[2] Consideration of when to do so is largely driven by performance bottlenecks.

and a "formula" or expression which in conjunction with the opcode describes the action to take. These may be arbitrarily composed to build programs.

The Nock specification is produced in full in Listing 2.1.

Listing 2.1: Nock 4K

```
Nock 4K

A noun is an atom or a cell. An atom is a natural number. A cell is an
ordered pair of nouns.

Reduce by the first matching pattern; variables match any noun.

nock(a)             *a
[a b c]             [a [b c]]

?[a b]              0
?a                  1
+[a b]              +[a b]
+a                  1 + a
=[a a]              0
=[a b]              1

/[1 a]              a
/[2 a b]            a
/[3 a b]            b
/[(a + a) b]        /[2 /[a b]]
/[(a + a + 1) b]    /[3 /[a b]]
/a                  /a

#[1 a b]            a
#[(a + a) b c]      #[a [b /[(a + a + 1) c]] c]
#[(a + a + 1) b c]  #[a [/[(a + a) c] b] c]
#a                  #a

*[a [b c] d]        [*[a b c] *[a d]]

*[a 0 b]            /[b a]
*[a 1 b]            b
*[a 2 b c]          *[*[a b] *[a c]]
*[a 3 b]            ?*[a b]
*[a 4 b]            +*[a b]
*[a 5 b c]          =[*[a b] *[a c]]

*[a 6 b c d]        *[a *[[c d] 0 *[[2 3] 0 *[a 4 4 b]]]]
*[a 7 b c]          *[*[a b] c]
```

```
*[a 8 b c]           *[[*[a b] a] c]
*[a 9 b c]           *[*[a c] 2 [0 1] 0 b]
*[a 10 [b c] d]      #[b *[a c] *[a d]]

*[a 11 [b c] d]      *[[*[a c] *[a d]] 0 3]
*[a 11 b c]          *[a c]

*a                   *a
```

The beginning of this specification is concerned with defining a noun, which is either an atom (a natural number) or a cell (an ordered pair of nouns). Several opcodes

Nouns are binary trees addressed using the natural numbers (in binary, the path down the tree is given by `0` for left and `1` for right) (see Figures 2.1 and 2.2).

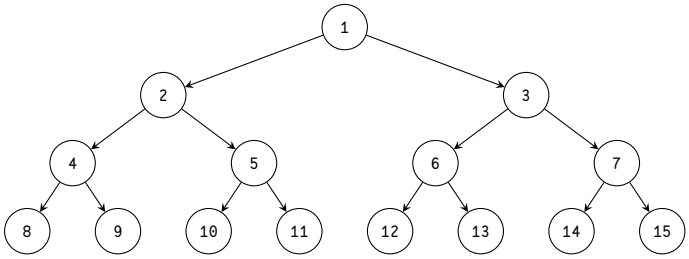

Figure 2.1: A binary tree with node addresses.

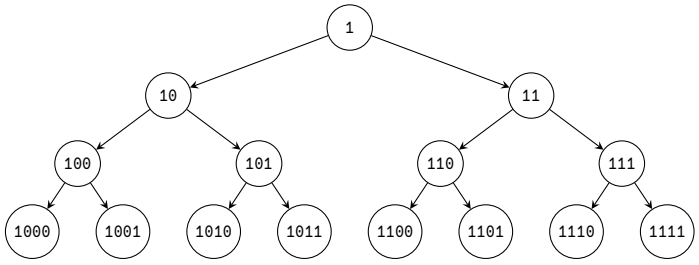

Figure 2.2: A binary tree with node addresses in binary indicating path.

A Nock system has the following characteristics:

1. Turing-complete: implements minimum requirements of μ-recursive functions. (This will be demonstrated below.)

2. Functional: uniquely maps `[subject formula]` to `product`. (This is a direct

consequence of the definitions above. Nock evaluation is pure and deterministic.)

3. Subject-oriented: subject entirely determines scope for formula. (This is a direct consequence of the definitions above.)

4. Homoiconic: uses same representation for data and control. (No distinction is made between code and data, in that both are nouns. Not all valid data is valid code, of course.)

5. Untyped: only knows about nouns. (While basic arithmetic is involved for cell addressing, no more algebraic machinery is imported.)

6. Solid-state: maintains no transient state in interpreter. (Each Nock evaluation is completely defined by its subject and formula. Noun 'mutability' is an apparent result of iterated evaluations of expressions on a subject.)

2.2.1 `I` Identity = Opcode 0

```
I x → x
```

Nock's binary tree addressing is fully general and yields the identity combinator `I` as a special case. (The cost, of course, is dragging in some machinery for basic arithmetic, as we will see—this is unnecessary in the base case but needed to describe and navigate nouns.)

Conceptually, we could write this in a hybrid form between combinators and Nock thus:

```
I x → *[x 0 1] ==
      /[1 x] ==
      x
```

In a sense, `I` resolves immediately to `[0 1]`, which is true but highlights a difference in terminology and argument structure. Nock conceives of itself as applying a formula to a subject, which in combinator terms means the subject is one argument and the formula(s) is/are another. However, the "position" of each will not be consistent, so you cannot simply map `x` to "formula".

Nock's homoiconicity, or the unification in representing code and data, renders it distinct in some ways. The slot operator `/` fas is more general than combinators typically are, as it assumes a structure to the input which is elided in combinatory logic. This is a key difference between Nock and combinatory logic: Nock is a language for manipulating structured data, while combinatory logic is a language for manipulating functions. The slot operator `/` fas is a way to access

parts of structured data, which is not typically part of combinatory logic. In our initial presentation, we are going to change the notation of subject and formula around to better match the conventions of the combinator definition for now.

2.2.2 K Constant = Opcode 1

```
K x y → x
```

The constant combinator K is implemented in Nock as opcode 1. It takes two arguments, a subject and a formula, and returns the formula unchanged, ignoring the subject. This is useful for building functions that always return a fixed value regardless of the input.

```
K x y → *[y 1 x] ==
        x
```

Note that we had to swap the order of x and y to match Nock's argument order. This is a common theme when translating between formal combinatory logic and Nock: the order of arguments may differ, so care must be taken to ensure that the correct values are being passed to the correct positions.

2.2.3 S Substitution = Opcode 2

```
S x y z → x z (y z)
```

The substitution combinator S is supplied in Nock as opcode 2, the evaluation operator. It takes three arguments, a subject and two formulas, and applies the first formula to the subject, then applies the second formula to the subject, and finally applies the result of the first application to the result of the second application.

```
S x y z → *[z 2 x y] ==
          *[*[z x] *[z y]]
```

2.2.4 The Axiomatic Operators

Because of Nock's cellular nature as a binary tree, we need the ability to access and manipulate nouns.

The noun equality operator = tis is used to implement to opcode 5, which takes two arguments and returns 0 if they are equal (when evaluated against the subject) and 1 if they are not.

```
:: TODO check
=[x y] → *[z 5 x y] ==
          =[*[z x] *[z y]]
```

Nock supplies this as a primitive operation, but it can be built out of combinators as well if we supply alternative conventions for boolean values. In SKI, we represent true as K (i.e., the first of two arguments) and false as SK (the second of two arguments).

Church Numerals in Nock

Classical combinatory logic does not build in the natural numbers, but derives them as Church numerals. Church numerals are a clever way to represent natural numbers as functions. They are more naturally expressed in the lambda calculus, but can be converted from λC to our preferred SKI basis. The Church numeral for zero is λf.λx.x, which means that it takes a function f and a value x, and returns x unchanged. The Church numeral for one is λf.λx.f x, which means that it takes a function f and a value x, and applies f to x once. The Church numeral for two is λf.λx.f (f x), which means that it takes a function f and a value x, and applies f to x twice, and so forth.

To build Church numerals in SKI, we can use the following translations from λC (presented without derivation from λC to SKI):

```
0 = K I
1 = I
2 = S (S (K S) K) I
3 = S (S (K S) K) (S (S (K S) K) I)
```

To cast Church numerals into Nock terms, let's apply our expressions for I, K, and S. (We face our next challenge in translating from SKI to Nock, which is that SKI evaluates from left to right, while Nock evaluates from right ("innermost") to left ("outermost").) We also need to distinguish a Church numeral from the natural number to which it corresponds.

```
0 = K I => *[y K I] == *[y 1 [0 1]] == [0 1]
1 = *[y I] == *[y 0 1] == /[1 y]
2 = *[z S I] == *[z 2 [0 1] [0 1]] == *[*[z [0 1]] *[z [0 1]]]
3 = *[z S (S (K S) K) I] == *[z 2 [2 [1 [0 3] [0 2]] [0 1]]]
    == *[*[z [2 [1 [0 3] [0 2]]]] *[z [0 1]]]
    == *[*[*[z 2 [1 [0 3] [0 2]]] *[z 1 [0 1]]] *[z [0 1
]]]
    == *[*[*[*[z 2 [0 3]] *[z 2 [0 2]]] *[z 1 [0 1]]] *[z [0 1]]]
    == *[*[*[*[*[z 2 [0 3]] *[z 2 [0 2]]] *[*[z 1 [0 1]]]] *[z [0 1]]]
    == *[*[*[*[*[z 2 [0 3]] *[z 2 [0 2]]] *[*[z 1 [0 1]]]] *[z [0 1]]
```

A general-purpose increment operator can be built in SKI as well. Let's call this increment operator `A` (classically written `Succ`) and derive it as follows:

```
::  S(S(KS)K)(S(KK)I)
```

Very awkward! Fortunately we don't need to continue down this road. We don't actually need Church numerals for Nock because Nock has built-in arithmetic, but they are a useful exercise for understanding how to build more complex combinators.

Increment, Opcode 4

Given that the natural numbers are built into Nock via both atoms and the tree addressing scheme, we can change an atom to its successor by applying the increment operator `+` (opcode 4). This takes a single argument, an atom, and returns the next natural number.[3]

Structure Checks, Opcodes 3 and 5

Two salient structure checks are provided in Nock: a cell check, which results in `TRUE/0` if the argument is a cell and `FALSE/1` otherwise (only an atom); and an equality check, which results in `TRUE/0` if the two arguments are equal and `FALSE/1` otherwise. These are implemented as opcodes 3 and 5, respectively.

There are not equivalent combinators in the `SKI` calculus for these checks since `SKI` does not know about any structure for its arguments.

(This pair of opcodes implies the possible desirability of a general-purpose structural check in Nock, which would match the overall binary tree layout of two nouns but not check the actual values of the atom leafs. Thus `[0 1 2]` would match `[0 1 2]` and `[0 1 3]`, but not `[[0 1] 2 3]` or `[0 1]`.)

2.2.5 Pythagoras and Peirce in Nock

> *Die ganzen Zahlen hat der liebe Gott gemacht, alles andere ist Menschenwerk.*
> The dear Lord created the natural numbers; all else is the work of Man.
> (Leopold Kronecker, 1886 lecture to Berlin Naturalists' Assembly)

> The first is that whose being is simply in itself, not referring to anything nor lying behind anything. The second is that which is what it

[3] This will always succeed for an atom, since every natural number has a successor. If the argument is a cell, it will fail, as `+[a b]` spins.

> is by force of something to which it is second. The third is that which is what it is owing to things between which it mediates and which it brings into relation to each other.
> (Charles Sanders Peirce, *Collected Papers* 2.356)

To take an esoteric aside on the nature of Nock as a computational platform, we may consider the Pythagorean relationship of the axioms and the opcodes.[4] Kronecker's quip was a bit of hyperbole leveled at Georg Cantor's number mysticism, but it nevertheless highlights the spartan utility of atoms and addressing in Nock.

The natural numbers are conventionally taken as a constructive ordered set of o and its successors along with addition, multiplication, and division (either floor division or division with remainder). (There is no additive inverse or subtraction in the natural numbers, since subtraction may result in a value that is not a natural number.) Nock dispenses with multiplication and division, relying strictly on addition in its axiomatic definitions. Nock o as an addressing system already in fact carries within it the seeds of the axiomatic operators `?` and `+`, because the addressed binary tree must know if a leaf is a cell or atom and be able to calculate its head and tail via use of the successor function `+`.

Furthermore, there is a sense in which computing is deeply semiotic. Semiotics is the philosophy of symbols and how they interrelate. (At the outside, you end up with language, culture, and thought itself as mutually recursive tangle of sign references, but we don't need to go that far here.[5]) Here we simply note that any noun always stands in relation to (at least) two possibilities: being a subject or being a formula. Of course, not every noun can be evaluated as a valid formula, but any noun can be utilized as a subject for appropriate formulas. Thus a noun is a member of an evaluation system: its purpose is to participate in a computation.[6]

Peirce's triadic semiotics gives us a way to conceive of nouns as embedded symbols. No noun is an island: it may be considered by itself, but it always bears a relationship to other nouns, even possible nouns. So it has both an identity and a contrast. Hearkening back to its role as a subject or possibly as a formula, a noun naturally carries Peirce's thirdness: it must stand in relation to other nouns.

Numbers as first explicated by Pythagoras remain magical when taken sufficiently seriously. Nock catches one more facet of their light.

[4] In a general way, `~barpub-tarber` and his Conk formulation as the field of possible Nocks (developed 2013–2017, republished 2025) parallel this analysis at the strict level of the opcodes.

[5] See the works of Saussure, Peirce, Eco, and Derrida, if you are interested in a great rabbit hole.

[6] Another stance of nouns to consider is with respect to the field of possible binary trees, that is, the field of possible nouns. We briefly note that the first possible formula is the crash or void, `[0 0]`, and the second possible formula is the identity, `[0 1]`.

2.3 Idioms & Design Patterns

Several of the combinators we examined above are not present in Nock *simpliciter*, but we can build them out of the other pieces. There are also common idioms which are baked in as "compound opcodes" (such as conditional branching, opcode 6).[7] Let's look at a few common combinators and then extend Nock with its standard opcdoes.

We can approach the construction of equivalent combinator expressions in Nock in two ways. The first is to start with the combinator definition in terms of `SKI` and then substitute our Nock expressions for `S`, `K`, and `I`. The second is to construct the simplest Nock expression, which utilizes the additional affordances of Nock (primarily slot addressing with opcode 0) to achieve the same effect.

2.3.1 `C` Flip Operator

```
C x y z == x z y
```

The flip operator `C` is a combinator that takes a pair of arguments and reverses their order. It is useful for building functions that take two arguments and apply them in a different order than they are given. In `SKI` terms, we can express this as follows:

```
C = S (S (K (S (K S) K)) S) (K K)
```

While we can formulate the flip operator `C` in terms of the Nock translation of `S` and `K`, it is far more practical to use opcode 0. This is almost free due to the binary tree nature of cells. In Nock, we express this as follows:

```
C x y z == x z y →
           [*[*[y z] [0 3] x] *[*[y z] [0 2] x]]
```

`C` simply rearranges the order of two nouns, which can be useful in managing conditional branches or modifying function argument order.[8] (For example, there is an operation called *currying* we'll see later which takes a function of two arguments and returns a function of one argument that applies the first argument to the second. If you wanted to bind the second argument, however, you could use something like `C` to flip the order of the arguments when you curry.)

[7] This first set of combinators derives from the original Schoenfinkel–Curry work (Curry, 1930).

[8] You can see here how using opcode 0 substantially simplifies the Nock equivalent expression versus the `SKI` definition.

2.3.2 B Composition Operator

```
B x y z == x (y z)
```

The composition operator B is a combinator that takes three arguments: a function and two values. It applies the second value to the third value, then applies the function to the result. In SKI terms, we can express this as follows:

```
B = S (K S) K
```

The more straightforward Nock expression for B in terms of S and K would be:

```
B x y z →
        *[z 7 x y] ==
        *[*[z y] x]
```

This is very similar to opcode 7, modulo rearranging the argument order.

B is useful for building functions that apply a function to the result of another function, allowing for more complex compositions of functions.

2.3.3 W Join Operator

```
W x y == x y y
```

The join operator W is a combinator that takes two arguments: a function and a value. It applies the function to the value twice; that is, it duplicates its second argument and applies it twice to the first. In SKI terms, we can express this as follows:

```
W = S S (S K)
```

The more straightforward Nock expression for W in terms of S and K would be:

```
W x y →
      *[*[y x] y]
      *[y *[y x]]
```

[*(42 [4 0 1]) 42]
This is very similar to opcode 8, modulo rearranging the argument order. Opcode 8 also has a built-in continuation to apply the result to a third argument.

2.3.4 U Duplication Operator

[9]

[9]The second set of combinators follows work done by Smullyan (1985) in his marvelous *To Mock a Mockingbird*.

2.4 Nock as Generally Computable

While it is beyond our scope to prove the equivalence of the four formulations of general computability, we can briefly demonstrate that Nock does satisfy the requirements of each.

`SKI` and `λC` completeness have been established above. Turing machine equivalence is established by the fact that we can build a universal Turing machine in Nock. Per * (*)Hopcroft1979, p. 148, a universal Turing machine has the following components:

- A finite, non-empty set of tape alphabet symbols, including a blank symbol.
- A finite, non-empty set of states, including a start state and one or more halting states.
- A transition function that, given the current state and the symbol being read, specifies the next state, the symbol to write, and the direction to move (left or right).

Nock as operator over nouns satisfies these requirements with the natural numbers (including crash `[0 0]` as blank) and cells to constitute nouns, while the transition function is represented by the evaluation rules of Nock itself.

Finally, Nock is generally recursive (or μ-recursive) because it has the necessary building blocks (**Paulsen1995** (**Paulsen1995**); Boolos, Burgess, and Jeffrey (2002), "6.2 Minimization", pp. 70–71):

- Constant function including zero (opcode 1)
- Increment or successor function (opcode 4)
- Parameter selection or variable access (opcodes 0 and 9 along with opcodes 7 and 8)
- Function composition or program concatenation (opcodes 7 and 8)
- Recursion or looping (opcode 0 with, e.g., opcodes 3, 5, and 6)

Thus Nock can express all μ-recursive functions.

2.5 A Stateful Nock Kernel

Nock is a crash-only pure language, which means at the base level it can only succeed and yield a new noun: no side effects, no exceptions, no state. Like a castaway on a desert island, we must build everything we need from raw materials at hand.

Nock has one trick built in and a couple of design patterns to help us get there:

1. Opcode 11 lets us send a hint to the runtime, which can be used to trigger side effects. The Nock expression formally doesn't know anything about these, but we can use them to print results, read input, and so forth.

2. The runtime can maintain state outside of Nock and supply it as a subject to subsequent Nock expressions. We will do a lot more with this later, but we want to highlight it as a key affordance now.

3. The virtualization pattern means that we can run Nock in Nock.

Opcode 11 can encode anything we want: the result of a hint is used by the runtime, and so anything the runtime is willing to work with can be produced.

The magic formula at the beating heart of a practical Nock operating system is simply:

```
*([state -.events] [9 2 10 [6 0 3] 0 2])
```

That is, apply the Nock formula against a subject pair consisting of the current state and the head of the list of events to process.

2.6 Virtualization

TODO derivation of Nock `+mock`

What could we want in a virtualized Nock environment?

Traditionally, opcode 12 has been used to permit the hyper-Turing "scry" operator, which allows you to look up a bound persistent value in a referentially transparent way. This is especially useful when a necessary value is not available in the current subject or needs to be supplied by the runtime (such as a system date or entropy).

We can also implement the combinators as extended Nock opcodes, aiding us in building more complex expressions from our higher-level language compiler when we get there.

13. `B` composition operator

14. `C` flip operator
15. `W` join operator
16. `U` duplication operator
17. `XXX`

https://web.archive.org/web/20070716223000/https://homepages.nyu.edu/ cb125/Lambda/ski.html
https://komiamiko.me/math/ordinals/2020/06/21/ski-numerals.html https://www.angelfire.com/tx4/cus/
https://www.sciencedirect.com/science/article/abs/pii/S0049237X99800172

Chapter 3

Interpreter Runtime

Abstract

Now that we have established a firm foundation for idiomatic programming in Nock, we can build an interpreter using a higher-level language to execute Nock formulas. At this point, we will build the simplest possible Nock interpreter, a tree-walking interpreter. This requires a cursor and some basic memory management but not much more. We will implement some minimalist static hints to permit side effects like output.

Chapter 4

Moon: A Higher-Level Language

Abstract

While somewhat more economical than a minimal combinator language, Nock is still quite low-level and cumbersome as we have seen. In this chapter, we will introduce a higher-level language called *Moon* which compiles to a Nock noun.[1]

4.1 Relationship of Moon to Nock

You can simply understand our goal as to produce executable Nock nouns using a more legible and flexible syntax than pure Nock affords. But it's worth casting Moon in the context of Nock's evaluation model.

The Moon compiler is a Nock noun which (as subject) operates on a text string to produce a new Nock noun. This second noun is itself a formula against which well-structured formulas may be evaluated. Formally,

```
Moon(source) == Nock(source moon-formula)
```

that is, the evaluation of the Moon compiler as a subject against a source string produces a Nock noun which is the compiled output of the Moon compiler. We can think of this as a two-step process: first, we compile the source code to a Nock noun, and then we evaluate other Nock nouns against that noun.

where `moon-source` is a text string containing the Moon source code, and `moon-formula` is a Nock noun which is the compiled output of the Moon compiler. The

[1] The name *Moon* is a double entendre: it is a micro-*Hoon*, the primary systems language of Urbit and NockApp applications, and it references Kleene's μ-recursive functions.

Moon compiler is itself a Nock noun, and so we can think of it as a self-compiling compiler.[2]

We will have to construct Moon in two stages: a launch stage ("New Moon") which can only compile simple expressions, a complex construction exemplifying all that we've seen; and a payload stage ("Crescent Moon") which can compile itself and thus becomes a Nockhound's flywheel ("Full Moon" being the completed product). We have two possible ways to construct New Moon as the launch stage: either we can write the Moon compiler directly in bespoke Nock by hand, or we can compose it in the same language as our interpreter, Rust.

4.2 ☒ New Moon

4.2.1 Design Criteria

New Moon is complete when it can compile some set of simple expressions to Nock. It does not yet, however, need to compile itself, which we defer until Crescent Moon.

Since Nock possesses its own set of idioms (as we've worked out in the preceding chapters), we know what the Nock products need to look like. A higher-level language can take into account the structure of its target ISA in its syntax or not, but we will find it easier to write Moon if we do. (Hoon takes this road, being essentially a macro language and type system over Nock.)

For instance, in Nock we would apply a function by loading the reference via opcode 9 and then replacing its sample with the new arguments. In Moon, we need a way to express this intent in an unambiguous way; let's say that we take a page out of Lisp's book and use parentheses to indicate function application. Thus, we could write a hypothetical Moon expression like this:

```
(+ 1 41)
```

That's reasonable if Lisp-flavored; indeed, we will find that Moon has Lisp-like characteristics. From a parsing standpoint, this needs to result in an abstract syntax tree (AST) which describes the operation and its arguments, thus detecting the head of the expression as its function and the tail to the ending parenthesis as arguments. We'll use a syntax in which text is marked by a `%` cen prefix, resulting in the hypothetical AST:

```
[%add 1 41]
```

[2] We here follow the exposition of **Yarvin2010c** (**Yarvin2010c**) in his description the a high-level language compiler to Nock for the first time.

and ultimately in the Nock noun (modulo tree addresses):

```
[8 [9 cor 0 arm] 9 2 10 [6 1 1 41] 0 2]
```

(Read it carefully—we pin the core locally with an opcode 8, then we modify the local copy with opcode 10, replacing the sample with the new arguments.)

You can follow a thread of logic from the initial expression in Moon to the final Nock noun, mediated by the AST representation. New Moon thus has two phases: a parser and a compiler. We can build them separately, but first let's look at more examples and their corresponding AST representations.

4.2.2 New Moon Examples

```
::  This is a comment.  It doesn't affect the AST or code.

::  This is a simple expression.
(+ 1 41)

::  This is a more complex expression.
(- 54 (+ 3 9))

::  This is a variable declaration.
(= a 42)

::  This is a loop with a check.
for (= a 0)
=a (- )
```

4.2.3 Parsing New Moon

We will soon run out of symbols: the punctuation in ASCII is limited. We could choose to switch to words throughout or to permit paired punctuation; we will do both eventually but for now we'll let it ride.[3]

4.2.4 Compiling New Moon

4.3 ☒ Crescent Moon

The objective of Crescent Moon is to bootstrap, or compile itself. Once we have this capability, expanding Moon to a full language becomes simply a matter of

[3] Another alternative is to support Unicode characters, as Julia does, but parsing UTF-8 text is more complex than Moon will support out of the box.

adding more syntax and semantics to the compiler. This means that Crescent Moon must be able to parse and compile text, with any required types being supplied as well.

At this point the question of the target ISA becomes acute: should Moon compile to strict Nock or to our extended Mock? That is, if our compiler utilizes either extended opcodes 6–9 or Mock opcodes 12–XX, how will it relate to the runtime system? At the end of the day, Nock is Nock (since we can always reduce), but we must make an aesthetic design choice which may have deep implications for Moon.

Let's consider what Hoon does. Hoon actually targets Mock (with opcode 12 only) and does some sensible reductions and compile-time checks. For instance, Hoon compilation of opcode 6 can eliminate branches if the conditional reduces to an opcode 1 (see `+cond`). The runtime compiler and system can make further optimizations: opcode 11 `%fast` hints for jets are only checked for opcode 9 expressions, not for opcode 2. Nock 7 is compiled away entirely.

At this point, we are not interested in optimizations, but there's no reason to forswear their future possibility. We will therefore make the following design decision: Moon targets canonical Nock without Mock opcodes (12 and higher). This will make Moon somewhat less efficient than Hoon as a systems language, but it will remain more legible.

4.4 ☒ Full Moon

From Crescent Moon, building Full Moon becomes a matter of expanding the language syntax, elaborating more parsers and other tools, and producing libraries of useful functions.

(at this point, back out from parentheses to centis for function application)

vase mode and type system redux

What other languages compile to Nock? The earliest higher-level language was Watt, the ancestor of Hoon. Hoon Jock

currying etc.

Chapter 5

A Virtual Machine Runtime

Abstract

To date, we have run Nock either by hand or using a simple interpreter. While this suffices for relatively small programs, our objective of building a Nock computer requires an "operating function", or persistent event loop. This chapter introduces the considerations of running Nock continuously and reactively as well as how to maintain state and respond to opcode 11 hints.

5.1 The Event Loop

A single Nock expression as a subject–formula pair accepts two nouns and yields a new noun. We can think of this new noun as the total state of a program or an operating system, feeding it into the next expression as the subject for a new formula to evaluate against. This preserves the determinism and legibility of Nock while allowing for complex state transitions.

5.1.1 A Simple State Machine

A state machine is a mathematical abstraction for a system that has a definite state at any particular time, and which changes in response to inputs.

Example: Stoplight

Think of a stoplight, which has a state of red, yellow, or green, and which changes its state in response to an external timer or a button press.

Example: Revolver

Or again, of a six-shot revolver, which if we include chamber order has 64 finite state permutations with thirteen possible inputs—to "load" or "clear" at each chamber and to "fire"—and two possible outputs—a "bang" or a "click" accompanied by a new state.

Aggregate or macro actions are also available—a "moon clip" corresponds to loading all six chambers at once, and a "dump cylinder" corresponds to clearing all six chambers at once. The revolver is a state machine with a finite number of states, inputs, and outputs.

[1]

We will call a state machine which adheres to the definition of its lifecycle function a *kernel*. Thus a kernel is a state machine of a certain shape which accepts nouns of a certain shape as input and produces nouns of a certain shape as output. The kernel is the event loop and some wrapper of library functionality to bound and define the complete behavior of the state machine. A kernel may be batch or continuous depending on whether its lifecycle function is reentrant.

5.1.2 An Evolved State Machine

While the two-armed model provides a straightforward way to manage state transitions, Nockhounds have found in practice that it is more convenient to supply four arms at the following axes:

`+4`: `+load` is used in kernel upgrades, allowing the event loop to update itself live.

`+10`: `+wish` accepts a core and parses it against a standard library for the kernel.

`+22`: `+peek` grants read-only access to the system's state; this is often called a "scry".

`+23`: `+poke` accepts events and processes them, resulting in a new state noun.

[1] `+poke` and `+peek` derive from a very old convention in `BASIC`, wherein `POKE` and `PEEK` commands allowed programmers to write to and read from arbitrary memory addresses.

Bibliography

Böhm, Corrado and Giuseppe Jacopini (1966). "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules." In: *Communications of the ACM* 9.5, pp. 366–371. ISSN: 0001-0782. DOI: 10.1145/355592.365646.

Boolos, George, John Burgess, and Richard Jeffrey (2002). *Computability and Logic*. 4th. Cambridge: Cambridge University Press. ISBN: 9780521007580.

Curry, Haskell (1930). "Grundlagen der kombinatorischen Logik." In: *American Journal of Mathematics* 52.3, pp. 509–536, 789–834. ISSN: 0002-9327. DOI: 10.2307/2370619.

Hopcroft, John E. and Jeffrey D. Ullman (1979). *Introduction to Automata Theory, Languages, and Computation*. Reading, MA: Addison-Wesley.

Kleene, Stephen Cole (1952). *Introduction to Metamathematics*. Amsterdam: North-Holland.

Smullyan, Raymond (1985). *To Mock a Mockingbird and Other Logic Puzzles: Including an Amazing Adventure in Combinatory Logic*. New York: Knopf.